

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Informatyki

Praca dyplomowa magisterska

**ANALIZA RUCHU SIECIOWEGO APLIKACJI MOBILNYCH NA
PLATFORMIE ANDROID**

Mikołaj Kmieciak, nr albumu 147575

Promotor
dr inż. dr inż. Michał Kalewski

POZNAŃ 2025

Streszczenie

Przedmiotem pracy jest analiza ruchu sieciowego generowanego przez wybrane aplikacje mobilne systemu Android. Zaimplementowano środowisko przechwytywania (mitmproxy, tcpdump), normalizacji i klasyfikacji z uwzględnieniem protokołów (HTTP/1.1, HTTP/2, QUIC), szyfrowania TLS oraz aspektów prywatności (domeny trackingowe, identyfikatory). Wyniki obejmują statystyki protokołów, charakterystykę ruchu według kategorii aplikacji oraz ocenę ryzyk potencjalnego ujawniania metadanych.

Słowa kluczowe: Android, analiza ruchu, TLS, QUIC, prywatność, mitmproxy.

Abstract

This thesis investigates network traffic generated by selected Android applications. A modular environment for traffic interception (mitmproxy, tcpdump), normalization and classification was implemented, focusing on protocols (HTTP/1.1, HTTP/2, QUIC), TLS encryption and privacy aspects (tracking domains, identifiers). Results include protocol usage statistics, traffic characteristics across app categories, and an assessment of potential metadata exposure risks.

Keywords: Android, network traffic analysis, TLS, QUIC, privacy, mitmproxy.

Spis treści

1	Wstęp	7
1.1	Motywacja	7
1.2	Stan wiedzy	9
1.2.1	Analiza ruchu sieciowego	9
1.2.2	Inżynieria wsteczna i testy penetracyjne	9
1.2.3	Prywatność, uprawnienia i trackery	9
1.2.4	Zużycie zasobów i kontekst behawioralny	9
1.2.5	Podsumowanie	9
1.3	Cel i zakres pracy	9
2	Podstawy teoretyczne	11
2.1	Architektura systemu Android	11
2.2	Model bezpieczeństwa	11
2.3	Protokoły	11
2.4	Techniki analizy ruchu	12
2.5	Prywatność i tracking	12
3	Metodyka badań	13
3.1	Pytania badawcze i hipotezy	13
3.2	Zakres i dobór próby	13
3.3	Próba badawcza: lista aplikacji	13
3.4	Badane elementy (checklista/metryki)	14
3.4.1	Komunikacja i protokoły	14
3.4.2	Prywatność i strony trzecie	14
3.4.3	Zachowanie w tle i zużycie zasobów	14
3.4.4	Logi i artefakty	14
3.4.5	Powiązanie z narzędziami	15
3.5	Środowisko badawcze	15
3.5.1	Sprzęt i oprogramowanie	15
3.5.2	Bezpieczeństwo, etyka i zgodność	15
3.6	Procedura zbierania danych	15
3.6.1	Przechwytywanie przez proxy	15
3.6.2	Rejestracja pakietów	15
3.6.3	Postępowanie z pinningiem	15
3.7	Normalizacja i klasyfikacja ruchu	16
3.8	Metryki i kryteria oceny	16
3.8.1	Bezpieczeństwo transmisji	16

3.8.2	Prywatność i uprawnienia	16
3.8.3	Zużycie zasobów	16
3.9	Plan eksperymentów	16
3.10	Analiza danych	17
3.11	Ograniczenia	17
3.12	Replikowalność	17
4	Architektura rozwiązania	19
4.1	Cel rozdziału	19
4.2	Przegląd architektury	19
4.3	Topologia sieci i ścieżki danych	19
4.4	Komponenty	20
4.4.1	Urządzenia testowe	20
4.4.2	AP/brama i segmentacja	20
4.4.3	Warstwa instrumentacji (opcjonalnie)	20
4.5	Przepływ danych i artefakty	20
4.5.1	Rejestracja i normalizacja	20
4.5.2	Integracja z pomiarami zasobów	21
4.6	Decyzje projektowe i kompromisy	21
4.6.1	Intercept vs. pcap	21
4.6.2	Wymuszanie fallbacku z QUIC	21
4.6.3	Obsługa pinningu	21
4.7	Bezpieczeństwo, etyka i izolacja	21
4.8	Ograniczenia architektury	21
4.9	Podsumowanie	21
5	Pierwsze spostrzeżenia przed badaniem właściwym	23
5.1	Inwentaryzacja aplikacji po resecie	23
5.2	Przechwytywanie radiowe Wiresharkiem w trybie monitorowania (niepowodzenie)	24
5.3	MITM przez proxy	24
5.4	MITM z instrumentacją (Frida) wobec pinningu — próba nieudana	24
5.5	Rejestracja ruchu po stronie urządzenia (PCAP/PCAPNG) — przykład Instagram (sukces)	25
6	Eksperyment: analiza ruchu aplikacji Instagram z użyciem PCAPdroid	27
6.1	Cel i założenia	27
6.2	Środowisko i narzędzia	27
6.3	Procedura (pojedyncza iteracja)	27
6.4	Zebrań dane (ta sesja)	28
6.5	Wyniki ilościowe	28
6.5.1	Wolumen i liczność wg protokołu	28
6.5.2	Największe cele (IP/port) wg wolumenu	28
6.6	Obserwacje jakościowe	29
6.7	Dyskusja i ograniczenia	29
6.8	Wariant porównawczy (HTTP/3 → HTTP/2)	29
6.9	Artefakty i replikowalność	29
6.10	Podsumowanie	29

7	Aspekty bezpieczeństwa i prywatności	31
7.1	Szyfrowanie i wersje TLS	31
7.2	Certificate pinning	31
7.3	Potencjalne wycieki metadanych	31
7.4	Domeny trackingowe	31
7.5	Implikacje	31
8	Dyskusja	33
8.1	Interpretacja wyników	33
8.2	Porównanie z literaturą	33
8.3	Zagrożenia dla ważności	33
8.4	Perspektywa praktyczna	33
9	Wnioski i dalsze prace	35
9.1	Podsumowanie	35
9.2	Odpowiedzi na pytania badawcze	35
9.3	Możliwe rozszerzenia	35
	Bibliografia	37

Rozdział 1

Wstęp

1.1 Motywacja

Rynek aplikacji mobilnych stał się dominującym kanałem konsumpcji treści i usług cyfrowych. Dane o penetracji dostępu mobilnego (ok. 71% globalnej populacji posiada subskrypcję mobilną do 2023 r.) [4] oraz statystyki udziału ruchu WWW (ponad 50% globalnych *page views* pochodzi z urządzeń mobilnych) [7] pokazują, że to właśnie środowisko mobilne jest dziś kluczowym miejscem, w którym kształtują się wzorce komunikacji sieciowej i materializują ryzyka dla prywatności.

W ekosystemie Androida współlistnieje wiele bibliotek i zestawów SDK firm trzecich — od analityki i reklam, przez personalizację, po rozwiązania antyfraudowe. Każda z tych warstw wnosi własne punkty końcowe, protokoły i mechanizmy telemetrii, często działające poza świadomością użytkownika, a niekiedy również poza pełną kontrolą zespołów deweloperskich aplikacji. W efekcie rośnie złożoność oraz nieprzejrzystość przepływów danych.

Konsekwencją tego stanu rzeczy są konkretne wyzwania badawcze:

- a) uzyskanie przejrzystości, *jakie* protokoły, domeny i dostawcy są realnie wykorzystywani przez aplikacje,
- b) ocena ryzyka ekspozycji metadanych (np. hosty, czasy, rozmiary transferów) mimo powszechnego szyfrowania treści,
- c) porównywalność zachowań komunikacyjnych między kategoriami aplikacji (komunikatory, multimedia, finanse, gry),
- d) budowa powtarzalnych narzędzi i procedur, które nadążą za ewolucją protokołów (HTTP/2, HTTP/3/QUIC) oraz mechanizmami obronnymi (np. *certificate pinning*).

Wobec powyższego potrzebne jest uporządkowane, możliwie obiektywne podejście do *systematycznej* obserwacji i oceny ruchu sieciowego aplikacji: środowisko pomiarowe, które pozwala porównywać różne aplikacje i kategorie, zdefiniowany zestaw metryk (zarówno ilościowych, jak i jakościowych) oraz protokoły postępowania uwzględniające ograniczenia współczesnych technologii transportowych i zabezpieczeń. Tak rozumiana ramka badawcza stanowi bezpośrednią motywację dla niniejszej pracy.

Drugi wektor motywacyjny stanowi szybka adopcja nowych protokołów transportowych i warstw aplikacyjnych. QUIC (standaryzowany jako HTTP/3 [10]) przenosi część logiki transportu do przestrzeni użytkownika (UDP + TLS 1.3 [28]), co zmienia charakterystykę obserwowalnych cech przepływów: klasyczne estymatory opóźnień i retransmisji z poziomu TCP stają się mniej

bezpośrednie, a analiza pakietów jest utrudniona przez szyfrowanie większej liczby pól. W konsekwencji wcześniejsze proporcje protokołów publikowane w literaturze mogą być nieaktualne dla obecnych wersji popularnych aplikacji.

Kolejnym aspektem jest prywatność i otoczenie regulacyjne. RODO (GDPR), a także DMA/DSA w UE akcentują minimalizację i przejrzystość przetwarzania. Architektura „multi-endpoint” (API własne + analityka + reklamy + antyfraud + CDN/edge) prowadzi do gęstych, równoległych zestawów połączeń tuż po interakcji użytkownika. Nawet przy pełnym szyfrowaniu treści korelacja metadanych (częstotliwość, rozmiary odpowiedzi, domeny) może ujawniać wzorce użycia funkcji (pattern-of-life). Transparentność takich wzorców jest ograniczona zarówno dla użytkownika, jak i audytorów. Narzędzia systemowe dają uogólniony obraz, a techniki inwazyjne (hooking, dynamic instrumentation) bywają blokowane przez mechanizmy anty-tampering.

Istnieje także motywacja praktyczna stworzenia lekkiego, powtarzalnego środowiska badawczego balansującego między:

- **Zasięgiem obserwacji** (kompletność metadanych HTTP[S]),
- **Inwazyjnością** (brak konieczności rootowania, minimalne modyfikacje),
- **Zgodnością prawną i etyczną** (anonimizacja identyfikatorów, brak utrwalania treści wrażliwych).

Istniejące narzędzia (np. *mitmproxy*) zapewniają bazowy przechwyt, ale wymagają rozszerzeń o:

- **Normalizacja** – ujednolicenie formatu danych.
- **Klasyfikacja** – kategoryzacja ruchu sieciowego.
- **Agregacja** – zbieranie i analiza metryk.
- **Automatyzacja** – skrypty do interakcji z aplikacjami.
- **Anonimizacja** – ochrona danych osobowych.

Z naukowego punktu widzenia identyfikowane luki obejmują:

- L1** Brak bieżących, empirycznych pomiarów dystrybucji HTTP/1.1 vs HTTP/2 vs QUIC w popularnych aplikacjach.
- L2** Ograniczone porównania między kategoriami aplikacji a zachowaniami komunikacyjnymi.
- L3** Niedostatek jawnych metryk ryzyka prywatności bazujących wyłącznie na metadanych.
- L4** Fragmentaryczne dane o wpływie certificate pinning na pokrycie przechwyty.

Interesariusze:

- **Użytkownik** – zyskuje pośrednio poprzez privacy-by-design.
- **Deweloper** – możliwość audytu nadmiarowych zależności.
- **Audytor bezpieczeństwa** – metryki priorytetyzacji ryzyka.
- **Społeczność naukowa** – powtarzalny pipeline do dalszych badań.

1.2 Stan wiedzy

1.2.1 Analiza ruchu sieciowego

Badania pokazują, że ruch generowany przez aplikacje mobilne niesie znaczące wyzwania dla prywatności – nawet konfiguracje TLS nie chronią całkowicie, bo analizę można przeprowadzać na metadanych lub kanałach bocznych. W przeglądzie literatury poświęconym analizie ruchu mobilnego autorzy klasyfikują cele, miejsca przechwytywania oraz metody wykorzystania danych sieciowych do profilowania użytkowników [27]. Inne prace demonstrują, jak sieć może zostać wykorzystana do identyfikacji konkretnej aplikacji, nawet przy szyfrowaniu, wykorzystując uczenie maszynowe na podstawie cech pakietów [24]. Przykłady wykorzystania narzędzi takich jak *mitm-proxy* i *tcpdump* pokazują, jak można wychwycić i analizować zarówno HTTP jak i inne kanały sieciowe [13], [26].

1.2.2 Inżynieria wsteczna i testy penetracyjne

Inżynieria wsteczna aplikacji mobilnych, ułatwiająca badanie mechanizmów takich jak SSL pinning, jest szeroko stosowana. Wczesne prace wskazują na powszechne błędy konfiguracji SSL w aplikacjach Androidowych — ponad połowa analizowanych aplikacji korzystających z SSL miała błędne wdrożenia lub brak obsługi poprawnego pinningu [30]. W celu przeprowadzania dynamicznych testów bezpieczeństwa wykorzystuje się narzędzia takie jak *Frida*, *Drozer* czy *MobSF*, które wspierają zarówno analizę dynamiczną, jak i automatyzację [6], [15].

1.2.3 Prywatność, uprawnienia i trackery

Coraz więcej badań zwraca uwagę na skalę gromadzenia danych przez aplikacje mobilne. Systematyczne opracowania przeglądowe pokazują spektrum metod oceny prywatności (statycznych, dynamicznych i hybrydowych) oraz wskazują luki w zgodności z regulacjami [18]. Narzędzia takie jak *Exodus Privacy* umożliwiają analizę APK pod kątem wbudowanych trackerów i żądanych uprawnień [5]. Wyniki badań empirycznych pokazują, że aplikacje często żądają nadmiarowych uprawnień lub integrują zewnętrzne trackery reklamowe [2].

1.2.4 Zużycie zasobów i kontekst behawioralny

Nadmierne użycie zasobów (np. bateria, CPU) może wskazywać na obecność komponentów śledzących lub brak optymalizacji. Do pomiarów wykorzystuje się m.in. *Battery Historian* oraz *Android Profiler* [20], [21]. Chociaż ten aspekt jest rzadziej eksplorowany w literaturze naukowej, podkreśla się jego znaczenie w identyfikacji ukrytych mechanizmów aplikacji [29].

1.2.5 Podsumowanie

Przegląd literatury wykazuje, że istnieje bogata baza wiedzy w obszarach analizy ruchu sieciowego, inżynierii wstecznej, testów penetracyjnych oraz badania prywatności aplikacji mobilnych. Jednocześnie wciąż brakuje holistycznego podejścia, które łączyłoby te wszystkie elementy — co stanowi niszę, którą niniejsza praca ma ambicję wypełnić.

1.3 Cel i zakres pracy

Cel: Celem pracy jest opracowanie i ewaluacja środowiska badawczego umożliwiającego systematyczną analizę bezpieczeństwa i prywatności aplikacji mobilnych na system Android. Praca

koncentruje się na przechwytywaniu, klasyfikacji oraz ocenie ruchu sieciowego, a także na badaniu mechanizmów uwierzytelniania, uprawnień i zużycia zasobów.

Zakres pracy obejmuje:

- **Środowisko przechwytywania** – konfiguracja narzędzi takich jak *mitmproxy*, *Wireshark*, *tcpdump*, a także przygotowanie środowiska laboratoryjnego z wykorzystaniem emulatorów oraz zrootowanych urządzeń mobilnych.
- **Analiza ruchu sieciowego** – identyfikacja używanych protokołów (HTTP/1.1, HTTP/2, QUIC, TLS), badanie bezpieczeństwa transmisji danych (szyfrowanie, SSL pinning) oraz klasyfikacja przesyłanych informacji.
- **Reverse engineering i testy penetracyjne** – dekompilacja aplikacji z wykorzystaniem *APKTool*, *Jadx*, *Ghidra* oraz testy przy użyciu narzędzi takich jak *MobSF*, *Frida*, *Metasploit*.
- **Analiza uprawnień i prywatności** – badanie przydzielanych uprawnień, wykrywanie trackerów i komponentów reklamowych (*Exodus Privacy*, *App Inspector*) oraz ocena zarządzania danymi użytkowników.
- **Analiza zużycia zasobów** – pomiary obciążenia baterii i CPU aplikacji z użyciem *Battery Historian* oraz *Android Profiler*.
- **Eksperymenty** – testy na wybranych aplikacjach z różnych kategorii (m.in. komunikatory, aplikacje społecznościowe, bankowe, rządowe), analiza różnic w implementacji mechanizmów bezpieczeństwa.
- **Badanie wektorów ataków** – symulacja ataków typu *Man-in-the-Middle*, analiza przechwytywania danych lokalnych, scenariusze działania offline/online, testy odporności na automatyczne skrypty i boty.
- **Analiza logów i ścieżki audytu** – sprawdzenie, jakie dane aplikacje rejestrują w logach oraz ocena ryzyka wycieku informacji.

Rozdział 2

Podstawy teoretyczne

2.1 Architektura systemu Android

Android jest otwartą, wielowarstwową platformą opartą na jądrze Linuksa. Stos oprogramowania obejmuje m.in. jądro (kernel) z mechanizmami bezpieczeństwa (np. SELinux), warstwę HAL (Hardware Abstraction Layer), biblioteki natywne, środowisko uruchomieniowe ART oraz framework Java/Kotlin z zestawem API, na którym budowane są aplikacje i usługi systemowe [3], [17]. W nowszych wersjach platformy pojawiły się dodatkowe izolatory, takie jak *SDK sandbox* (separowany proces dla wybranych SDK), co jeszcze bardziej ogranicza wpływ komponentów zewnętrznych na proces aplikacji [25].

2.2 Model bezpieczeństwa

Fundamentem bezpieczeństwa Androida jest *piaskownica* (application sandbox): każda aplikacja otrzymuje unikalny UID i działa w odseparowanym procesie z izolowanym przestrzennie systemem plików. Dodatkową ochronę zapewnia polityka MAC (SELinux), a także model uprawnień ograniczający dostęp do wrażliwych zasobów (lokalizacja, kamera, mikrofon) – w tym uprawnienia żądane w czasie działania (*runtime permissions*) [1], [16], [23]. Łańcuch zaufania domykają mechanizmy podpisywania aplikacji, Verified Boot/AVB oraz proces weryfikacji w ekosystemie sklepu (Play Protect, audyty polityk), co Google cyklicznie raportuje [8]. W kontekście komunikacji sieciowej część aplikacji implementuje *certificate pinning*, które utrudnia badaczowi pasywne przechwycenie treści HTTPS przez klasyczne proxy – poprawnie wdrożony pinning ma powodować odrzucenie połączenia z certyfikatem wystawionym przez zaufane, lecz nieprawidłowe (dla danej domeny) CA [12].

2.3 Protokoły

Przeglądane aplikacje korzystają z rodziny protokołów HTTP w różnych wersjach oraz z TLS do uwierzytelnienia i poufności.

- **HTTP/1.1** – klasyczny, tekstowy protokół nad TCP (przykładowe różnice semantyczne i transportowe względem nowszych wersji omawiają poniższe punkty HTTP/2/HTTP/3).
- **HTTP/2** – binarne ramkowanie, kompresja nagłówek i *multiplexing* wielu strumieni na jednym połączeniu TCP w celu redukcji opóźnień [9].
- **HTTP/3** – mapowanie semantyki HTTP na QUIC (UDP), eliminujące wpływ HoL na poziomie transportu i skracające zestawienie połączenia [10], [22].

- **TLS 1.3** – domyślny protokół kryptograficzny dla HTTPS, skracający *handshake*, upraszczający zestaw szyfrów i wspierający 0-RTT (z zastrzeżeniami bezpieczeństwa) [28].

2.4 Techniki analizy ruchu

W analizach bezpieczeństwa mobilnego wykorzystuje się zwykle dwa komplementarne podejścia: (1) *przechwytywanie i inspekcję ruchu* oraz (2) *instrumentację/dynamiczną obserwację aplikacji*. Do przechwytywania pakietów i strumieni wykorzystywane są m.in. *tcpdump* i *Wireshark*, a do pośredniczenia w ruchu HTTP(S) – *mitmproxy* (instalacja zaufanego CA na urządzeniu w celu deszyfracji ruchu) [11], [14], [26], [31]. W przypadku aplikacji z poprawnym *certificate pinning* wymagane bywa podejście z instrumentacją (np. *Frida*) lub środowisko z większą kontrolą (urządzenie z dostępem *root*/emulator, własny trust store), przy czym nowoczesne techniki (np. niestandardowi *trust managerzy*, natywne weryfikacje, integracja z *Play Integrity*) istotnie podnoszą poprzeczkę dla badań dynamicznych [6], [12]. Wybór między emulatorem a urządzeniem fizycznym wpływa na wierność zachowania (emulator ułatwia kontrolę i automatyzację; urządzenie fizyczne lepiej oddaje ograniczenia sprzętowe i integracje OEM) [11].

2.5 Prywatność i tracking

Problem prywatności w aplikacjach mobilnych obejmuje zarówno zakres i uzasadnienie uprawnień, jak i obecność komponentów stron trzecich (SDK analityczne/reklamowe). Do statycznego wykrywania trackerów i przeglądu uprawnień w APK wykorzystywane są m.in. *Exodus Privacy* oraz narzędzia deweloperskie Androida [5], [16]. Równolegle platforma rozwija rozwiązania mające ograniczyć śledzenie międzyaplikacyjne (np. *Privacy Sandbox on Android*) [19]. W praktyce ocena prywatności powinna łączyć analizę statyczną i dynamiczną (ruch sieciowy, zachowanie w tle), a także kontrolować zużycie zasobów (bateria/CPU) – symptomy nadmiernej telemetrii mogą ujawniać się jako wzrost aktywności sieciowej w tle czy anomalia energetyczne.

Rozdział 3

Metodyka badań

3.1 Pytania badawcze i hipotezy

Celem metodyki jest rzetelne porównanie praktyk komunikacyjnych i mechanizmów ochrony prywatności w wybranych aplikacjach mobilnych na Androidzie. Odpowiadamy na pytania:

1. Jakie protokoły i wersje (HTTP/1.1, HTTP/2, HTTP/3/QUIC, TLS) wykorzystują badane aplikacje i czy połączenia są odpowiednio chronione [9], [10], [22], [28]?
2. Czy możliwy jest pasywny lub aktywny podsłuch ruchu z użyciem proxy/interceptora i jakie bariery występują (np. certificate pinning) [11], [12], [14]?
3. Jakie wzorce prywatności występują (trackery, punkty końcowe stron trzecich, zakres uprawnień) i jak korespondują z politykami platformy [5], [16], [19]?
4. Jaki jest profil zużycia zasobów (energia/CPU) w typowych scenariuszach użycia [20], [21]?

3.2 Zakres i dobór próby

Analizie podlegają popularne aplikacje z różnych kategorii (komunikacja, rozrywka, finanse, usługi publiczne). Dla każdej aplikacji realizujemy powtarzalne, zdefiniowane niżej scenariusze: uruchomienie, rejestracja/logowanie (jeśli dotyczy), przeglądanie treści, praca w tle (okno obserwacji), wylogowanie. Zakres nie obejmuje łamania zabezpieczeń serwerowych ani naruszeń regulaminów usług.

3.3 Próba badawcza: lista aplikacji

Na potrzeby badań przyjęto celowo dobraną próbę aplikacji mobilnych na Androidzie, obejmującą popularne kategorie (media społecznościowe, finanse, usługi publiczne, edukacja, handel/retail, rozrywka). Zestaw wyjściowy:

- **TikTok, Instagram, Facebook** — media społecznościowe;
- **Spotify** — streaming audio;
- **XTB** — finanse/inwestycje;
- **mObywatel** — usługi publiczne;
- **USOS** — system uczelniany/edukacja;

- **Posnania** — handel/galeria (aplikacja obiektu handlowego);
- **Voronoi, Gazet** — aplikacje lokalne/utility.

3.4 Badane elementy (checklista/metryki)

Ocena każdej aplikacji opiera się na zunifikowanej liście elementów, łączącej analizę ruchu, prywatności oraz zachowania w tle i zużycia zasobów:

3.4.1 Komunikacja i protokoły

- wersja protokołu aplikacyjnego i transportowego: HTTP/1.1, HTTP/2, HTTP/3/QUIC; identyfikacja po ALPN i charakterystyce strumieni;
- bezpieczeństwo transmisji: obecność TLS, wersja (w szczególności TLS 1.3), parametry sesji, błędy weryfikacji certyfikatu;
- *certificate pinning*: objawowa detekcja (odrzuć łańcucha, brak wglądu w treść po zainstalowaniu CA użytkownika); próby obejścia tylko w granicach zgodnych z zaleceniami badawczymi (instrumentacja, emulator/środowisko z własnym trust store);
- profil ruchu: liczność żądań, kody odpowiedzi, rozmiary, timingi; rozróżnienie pierwszego planu vs. tło; bursty i pory największej aktywności.

3.4.2 Prywatność i strony trzecie

- mapowanie domen/punktów końcowych i liczba unikalnych hostów/AS (proksy dla rozproszenia danych);
- identyfikacja trackerów/SDK reklamowych (analiza statyczna APK + weryfikacja dynamiczna w ruchu);
- korelacja uprawnień z obserwowanym zachowaniem (np. żądanie lokalizacji vs. transmisje geodanych); zgodność z dobrymi praktykami platformy;
- praktyki sesyjne: przechowywanie tokenów/ID, rotacja, wysyłka wrażliwych parametrów; ewentualne użycie plików cookie w komponentach webview.

3.4.3 Zachowanie w tle i zużycie zasobów

- aktywność sieciowa w tle (po zamknięciu/wylogowaniu, w oknie obserwacji);
- energia/CPU: metryki z narzędzi deweloperskich (timeline, wakelocki, sieć vs. czas) oraz anomalie energetyczne w korelacji ze zdarzeniami sieciowymi;
- scenariusze szczególne: tryb offline/online (spójność i bezpieczeństwo synchronizacji), zmiana łączności, restart aplikacji/urządzenia.

3.4.4 Logi i artefakty

- występowanie wrażliwych danych w logach/plikach tymczasowych; format i ochrona (szyfrowanie/dostęp);
- zgodność z zasadą minimalizacji (tylko niezbędne dane w logach oraz telemetrii).

3.4.5 Powiązanie z narzędziami

- przechwyt i inspekcja: mitmproxy (CA użytkownika), tcpdump/Wireshark (pcap);
- analiza statyczna/dynamiczna: Exodus Privacy, Frida/MobSF (w razie potrzeby);
- pomiary zasobów: Battery Historian, Android Profiler.

3.5 Środowisko badawcze

3.5.1 Sprzęt i oprogramowanie

Badania realizowane są w kontrolowanym labie z wydzieloną siecią bez dostępu osób trzecich. Jako urządzenia testowe wykorzystywane są telefony z Androidem (konfiguracje opisane w dzienniku badań) oraz host linuxowy pełniący rolę punktu dostępowego i kolektora ruchu. Intercepcję i rejestrację realizują: *mitmproxy* (CA użytkownika), *tcpdump/Wireshark*; infrastruktura sieciowa: *hostapd*, *dnsmasq* [14], [31].

Konfiguracja labu odzwierciedla praktyki rekomendowane przez OWASP MASTG dla przechwytywania ruchu HTTP(S) oraz alternatyw nie-HTTP [11]. W przypadkach aplikacji stosujących pinning rozważana jest instrumentacja dynamiczna (np. *Frida*) oraz uruchomienie w środowisku z większą kontrolą nad trust store (emulator, urządzenie z dostępem *root*) [6], [12].

3.5.2 Bezpieczeństwo, etyka i zgodność

Badania prowadzone są na własnych kontach testowych; nie gromadzimy wrażliwych danych osób trzecich. Zachowana jest minimalizacja danych oraz separacja środowisk. Wszelkie obserwacje interpretujemy przez pryzmat zasad ochrony danych (np. zgodność z wymogami RODO pozostaje poza zakresem eksperymentalnym, ale motywuje ocenę prywatności).

3.6 Procedura zbierania danych

3.6.1 Przechwytywanie przez proxy

Główną ścieżką jest ruch przez *mitmproxy* z zainstalowanym zaufanym CA użytkownika na urządzeniu. Dla każdej aplikacji rejestrujemy strumienie żądań i odpowiedzi, metadane (czas, metoda, kody, rozmiary), informacje o ALPN/SNI i właściwości połączenia TLS [11], [14]. Zwracamy uwagę na przypadki odrzucenia łańcucha certyfikatu (typowy objaw pinningu).

3.6.2 Rejestracja pakietów

Równolegle pozyskujemy zrzuty pakietów (*pcap*) z interfejsów sieciowych (*tcpdump/Wireshark*) dla analizy niskopoziomowej, klasyfikacji protokołów i weryfikacji QUIC/HTTP/3 na podstawie ALPN i charakterystyki strumieni [10], [22], [31].

3.6.3 Postępowanie z pinningiem

Jeżeli aplikacja implementuje certificate pinning poprawnie, atak MITM z CA użytkownika powinien się nie powieść [12]. W takich przypadkach dokumentujemy objawy (np. brak ruchu odszyfrowanego, błędy połączeń), a dalsze badanie może wymagać instrumentacji (hooking wybranych API) lub środowiska z modyfikowalnym trust store/emulacji [11], [12]. Zgodnie z MASTG,

obejście pinningu bywa możliwe, ale zależy od konkretnych implementacji i nie zawsze jest wykonalne bez głębszych modyfikacji aplikacji/OS [12].

3.7 Normalizacja i klasyfikacja ruchu

Po zebraniu danych normalizujemy je do poziomu *flow* (5-tuple + kierunek + ALPN) i *transakcji HTTP*. Ekstrahujemy następujące cechy:

- protokół transportowy i aplikacyjny (TCP/UDP; HTTP/1.1, HTTP/2, HTTP/3), wersja TLS, wynik weryfikacji certyfikatu [9], [10], [28];
- metryki ruchu (liczba żądań, kody odpowiedzi, rozmiary, czasy, bursty w tle/na pierwszym planie);
- informacje o domenach i dostawcach (SNI/Host), liczba unikalnych hostów/AS (proxymalna miara rozproszenia danych);
- sygnały prywatności (wywołania do znanych domen analitycznych/reklamowych, parametry identyfikatorów), triangulowane z wynikami analizy statycznej [5].

3.8 Metryki i kryteria oceny

3.8.1 Bezpieczeństwo transmisji

- udział ruchu szyfrowanego (TLS) i nieszyfrowanego (HTTP bez TLS);
- wersje TLS i negocjacje ALPN; wykrycie HTTP/3/QUIC [10], [22], [28];
- wskaźnik pinningu (objawowy): odsetek żądań, których nie udało się zdeszyfrować przy poprawnej konfiguracji CA użytkownika, z adnotacją o przyczynie [11], [12].

3.8.2 Prywatność i uprawnienia

- obecność trackerów i domen stron trzecich wskazana przez analizę statyczną APK (*Exodus Privacy*) oraz potwierdzona w ruchu dynamicznym [5];
- zakres i częstotliwość wywołań w tle do zewnętrznych punktów końcowych (okno obserwacji);
- mapowanie żądanych uprawnień na obserwowane zachowania sieciowe (np. dostęp do lokalizacji vs. transmisje geodanych) [16], [19].

3.8.3 Zużycie zasobów

- metryki z *Battery Historian*: aktywność komponentów, zdarzenia wakelock, sieć vs. czas, korelacje ze scenariuszami [20];
- metryki z *Android Profiler*: ślady CPU/pamięci/sieci w trakcie scenariuszy i pracy w tle [21].

3.9 Plan eksperymentów

Dla każdej aplikacji wykonujemy sekwencję:

1. **Przygotowanie**: czyste uruchomienie, weryfikacja uprawnień, instalacja CA użytkownika (jeśli dotyczy), reset liczników energii.

2. **Scenariusz interakcji:** rejestracja/logowanie, nawigacja po kluczowych widokach, generowanie typowego ruchu (pobieranie treści, wyszukiwanie), a następnie wyjście.
3. **Okno tła:** urządzenie pozostaje bez interakcji przez N minut w tej samej sieci; rejestrujemy ruch i zdarzenia energetyczne.
4. **Wylogowanie i zamknięcie:** kończymy sesję, sporządzamy checksumy artefaktów, archiwizujemy logi/profilery.

Każdy scenariusz powtarzamy co najmniej trzykrotnie i o różnych porach dnia, aby ograniczyć wpływ zmienności sieci i backendu.

3.10 Analiza danych

Zastosujemy statystyki opisowe (medianą i IQR dla zmiennych skośnych), wykresy czasowe i heatmapy przepływów. Dla wybranych porównań (np. porównanie intensywności wywołań w tle między kategoriami) stosujemy nieparametryczne testy różnic (np. Mann–Whitney/Kruskal–Wallis) z poprawką na wielokrotność, o ile rozkłady i liczebności na to pozwalają.

3.11 Ograniczenia

Ograniczenia obejmują: (i) zależność od wersji aplikacji i backendu w czasie pomiaru; (ii) niepełną obserwowalność w aplikacjach z rygorystycznym pinningiem/ochronami anty-tamper (*Play Integrity*, niestandardowi *trust managerzy*) [12]; (iii) wpływ środowiska (emulator vs. urządzenie fizyczne) na zachowanie [11]. Wyniki nie są uogólnieniem na całe kategorie, lecz charakterystyką zbadanych wersji.

3.12 Replikowalność

Zapewniamy spójne wersjonowanie artefaktów: konfiguracji *mitmproxy*, skryptów ekstrakcji cech, próbek *pcap* (zanonimizowanych) i raportów z *Battery Historian*. Tam, gdzie to możliwe, udostępniamy parametry komend i hashe binariów narzędzi (bez udostępniania samych APK). Dokumentujemy wersje protokołów i bibliotek (ALPN/TLS) [10], [22], [28].

TABELA 3.1: Mapa elementów badania do narzędzi i metryk

Element badania	Narzędzie / źródło	Przykładowe metryki / artefakty
Protokoły i wersje (HTTP/1.1, HTTP/2, HTTP/3/QUIC)	<i>tcpdump</i> , <i>Wireshark</i> [26], [31]	ALPN/SNI; rozróżnienie HTTP/1.1 vs HTTP/2 vs HTTP/3; cechy QUIC (DCID/SCID), liczba strumieni [9], [10], [22].
Bezpieczeństwo transmisji (TLS)	<i>Wireshark</i> , <i>mitmproxy</i> [14], [31]	wersja TLS (1.2/1.3), zestaw szyfrów, wynik walidacji certyfikatu, renegotiacje, 0-RTT (jeśli występuje) [28].
Intercepcja (proxy)	HTTPS <i>mitmproxy</i> , MASTG [11], [14]	odsetek żądań odszyfrowanych; metody, kody HTTP, rozmiary, nagłówki (<i>Authorization</i> , <i>Set-Cookie</i>); TTFB i czas całkowity.

Element badania	Narzędzie / źródło	Przykładowe metryki / artefakty
Certificate pinning (objawowo)	<i>mitmproxy</i> + logi; opcj. <i>Frida</i> [6], [12]	odsetek połączeń odrzuconych mimo zaufanego CA użytkownika; wzorce błędów/wyjątków; klasy API do hookowania.
Ruch w tle / bez interakcji	<i>mitmproxy</i> , <i>tcpdump</i> [14], [26]	żądania/min w tle; rozkład czasowy (bursty), domeny docelowe, interwały retry/backoff.
Semantyka HTTP	<i>mitmproxy</i> [14]	metody (GET/POST/PUT...), kody odpowiedzi, rozmiary, kolejność/priorytetyzacja strumieni (H2/H3).
Trackery / strony trzecie	Exodus Privacy + korelacja w PCAP [5]	lista trackerów/SDK; mapowanie domen→dostawcy; identyfikatory/parametry telemetryczne w żądaniach.
Uprawnienia a zachowanie	manifest + Android Docs [16]	zestaw żądanych uprawnień; mapowanie uprawnienie→zachowanie (np. lokalizacja→transmisje geodanych); spójność deklaracji.
Zużycie energii	Battery Historian [20]	wakelocki, aktywność sieć czas, top procesy, zdarzenia radiowe; korelacje ze szczytami ruchu.
CPU / pamięć / sieć	Android Profiler [21]	%CPU, zdarzenia GC, użycie pamięci, throughput sieci; ślady w scenariuszach i w tle.
Scenariusze offline/online i degradacje	<i>mitmproxy</i> , <i>tcpdump</i> [14], [26]	retry, błędy synchronizacji, fallback protokołu/transportu, spójność danych po reconnect.
Logi i artefakty lokalne	<i>adb logcat</i> , inspekcja plików	obecność wrażliwych danych w logach/plikach tymczasowych; rotacja i ochrona (szyfrowanie/dostęp).

Rozdział 4

Architektura rozwiązania

4.1 Cel rozdziału

Celem rozdziału jest opis architektury środowiska badawczego, w którym realizowana jest analiza bezpieczeństwa i prywatności aplikacji mobilnych na Androidzie. Rozdział definiuje topologię sieci, komponenty, przepływ danych oraz decyzje projektowe umożliwiające analizę HTTP/1.1, HTTP/2 i HTTP/3/QUIC z uwzględnieniem TLS 1.3 i barier takich jak *certificate pinning* [9]–[12], [22], [28].

4.2 Przegląd architektury

Środowisko składa się z czterech głównych bloków (rys. ??):

1. **Urządzenie testowe (DUT)** – telefon z Androidem wykonujący scenariusze testowe.
2. **Brama/AP (Linux)** – laptop z Debianem pełniący rolę punktu dostępowego (Wi-Fi) i bramy NAT; działa na nim *hostapd*, *dnsmasq*, *mitmproxy*, *tcpdump*/*Wireshark*.
3. **Warstwa przechwytywania** – intercept HTTPS przez *mitmproxy* (CA użytkownika) oraz niezależny zrzut pakietów (*pcap*) dla niskiego poziomu [14], [26], [31].
4. **Repozytorium i przetwarzanie** – magazyn *pcap/flow*, logi proxy i skrypty normalizacji do metryk opisanych w rozdziale „Metodyka”.

4.3 Topologia sieci i ścieżki danych

Linuxowy host uruchamia *hostapd* (AP) oraz *dnsmasq* (DHCP/DNS). Ruch z DUT przechodzi przez bramę, gdzie:

1. dla HTTP/HTTPS po TCP – jest przekierowywany do *mitmproxy* (intercept), co umożliwia inspekcję transakcji HTTP, nagłówków i treści [11], [14];
2. niezależnie – interfejsy są rejestrowane *tcpdump*/*Wireshark* w celu analizy ALPN/SNI, wersji TLS, identyfikacji QUIC/HTTP/3 i korelacji strumieni [10], [22], [26], [31].

Ponieważ **HTTP/3 działa nad QUIC (UDP)**, nie może być transparentnie odszyfrowany przez klasyczne *TLS MITM*. W praktyce stosowane są dwa tryby: (i) analiza metadanych QUIC/HTTP/3 z *pcap* oraz (ii) *fallback* do HTTP/2 wymuszony polityką sieci (np. blokada/limitacja UDP/443) – zgodnie z dobrymi praktykami badawczymi i etyką testów [10], [11].

4.4 Komponenty

4.4.1 Urządzenia testowe

Dwa urządzenia z Androidem pełnią rolę DUT. Konfiguracja badawcza przewiduje:

- czyste profile i kontrolowane konto testowe;
- instalację zaufanego CA użytkownika dla *mitmproxy* (o ile aplikacja nie stosuje pinningu);
- ograniczenie danych w tle aplikacji niewymagających tego dla poprawnego działania (minimalizacja tła w oknie obserwacji);
- wariant z uprawnieniami *root*/emulator w przypadku analiz wymagających pełnej kontroli trust store lub instrumentacji [11], [12].

4.4.2 AP/brama i segmentacja

Host z Debianem działa jako AP i brama NAT. *hostapd* odpowiada za nadawanie SSID i parametry bezpieczeństwa Wi-Fi, *dnsmasq* – za DHCP/DNS, a routing (*iptables/nftables*) rozdziela podsieci i wymusza ścieżki pomiarowe. Na hoście działają równolegle:

1. **Proxy** – *mitmproxy* w trybie intercept z CA użytkownika (generowanie certyfikatów dla domen), co umożliwia analizę treści HTTP/1.1 i HTTP/2 [11], [14];
2. **Kolektor pakietów** – *tcpdump*/*Wireshark* zapisujące strumienie *pcap* z interfejsów (Wi-Fi i uplink) do analizy QUIC/TLS [22], [26], [28], [31].

4.4.3 Warstwa instrumentacji (opcjonalnie)

Dla aplikacji z rygorystycznym *certificate pinning* (własne **TrustManager**, weryfikacje natywne) stosuje się instrumentację (np. *Frida*) w celu hookowania wybranych API. W wielu komercyjnych aplikacjach obejście pinningu bez *roota* bywa niewykonalne lub znacząco utrudnione; w takich przypadkach badanie ogranicza się do metadanych sieciowych i efektów objawowych [6], [12].

4.5 Przepływ danych i artefakty

4.5.1 Rejestracja i normalizacja

Dane zebrane w sesji obejmują:

- **transakcje HTTP** z *mitmproxy*: metoda, kod, nagłówki, rozmiary, czasy, treści (gdy brak pinningu);
- **zrzuty pcap**: pełne pakiety/strumienie do analizy ALPN/SNI, wersji TLS, cech QUIC (DCI-D/SCID), liczby strumieni, retransmisji [9], [10], [22];
- **logi i metadane** z hosta/AP: reguły routingu, kroki wymuszenia fallback, znaczniki czasu.

Artefakty są następnie normalizowane do poziomu *flow* i *transakcji HTTP*, a następnie agregowane do metryk zdefiniowanych w metodyce (udział szyfrowania, wersje protokołów, wskaźnik pinningu, aktywność w tle, liczba unikalnych domen/AS, itp.).

4.5.2 Integracja z pomiarami zasobów

Dla wybranych sesji generowane są artefakty energetyczne i wydajnościowe (np. `bugreport` dla *Battery Historian*, ślady *Android Profiler*) w celu korelacji szczytów ruchu z użyciem zasobów [20], [21].

4.6 Decyzje projektowe i kompromisy

4.6.1 Intercept vs. pcap

Intercept przez *mitmproxy* daje największą widoczność semantyki HTTP, lecz jest blokowany przez *certificate pinning* oraz nie obejmuje QUIC/HTTP/3. Zrzut `pcap` jest niezależny od pinningu i pozwala na identyfikację QUIC/HTTP/3, ale ogranicza się do metadanych i analizy transportowej. Z tego powodu architektura łączy obie ścieżki [10], [11], [14].

4.6.2 Wymuszanie fallbacku z QUIC

Aby porównać semantykę HTTP/2 i HTTP/3 dla tej samej aplikacji, można tymczasowo ograniczyć UDP/443 (kontrolowany eksperyment) i odnotować wpływ na wydajność i ścieżki domenowe. Takie działanie jest wykonywane wyłącznie w sieci laboratoryjnej i na kontach testowych [10], [11].

4.6.3 Obsługa pinningu

Gdy aplikacja odrzuca certyfikat CA użytkownika, raportujemy to jako przypadek pinningu i przechodzimy do analizy warstwy transportowej (ALPN/SNI, docelowe domeny, częstotliwość wywołań) oraz — jeśli uzasadnione — do analizy instrumentacyjnej/emulatora. Decyzja o dalszych krokach zależy od wpływu na wyniki i kosztu badania [6], [12].

4.7 Bezpieczeństwo, etyka i izolacja

Lab działa w wydzielonej podsieci, bez dostępu osób trzecich; używane są wyłącznie konta testowe i minimalny zestaw danych. Zasada minimalizacji dotyczy również logów: przechowujemy tylko to, co niezbędne do replikowalności i analizy wniosków. *mitmproxy* i `pcap` są rotowane i anonimizowane, a artefakty opisane checksummami.

4.8 Ograniczenia architektury

- Aplikacje z rygorystycznym pinningiem lub natywnymi weryfikacjami mogą uniemożliwić wgląd w treść; wtedy pozostaje analiza metadanych transportowych [12].
- QUIC/HTTP/3 nie jest odszyfrowywalny przez klasyczny MITM; wymaga analizy `pcap` lub wymuszenia fallbacku [10].
- Wyniki zależą od wersji aplikacji/backendu oraz od konfiguracji urządzenia (root/emulator vs urządzenie produkcyjne).

4.9 Podsumowanie

Architektura łączy **dwie komplementarne ścieżki obserwacji**: pełny intercept HTTP(S) tam, gdzie to możliwe, oraz niezależną rejestrację pakietów dla QUIC/HTTP/3 i przypadków z

pinningiem. Takie połączenie maksymalizuje obserwowalność przy zachowaniu zgodności z dobrymi praktykami badań bezpieczeństwa mobilnego [9]–[12], [14], [22], [26], [28], [31].

Rozdział 5

Pierwsze spostrzeżenia przed badaniem właściwym

5.1 Inwentaryzacja aplikacji po resecie

Cel: uzyskanie stanu wyjściowego urządzeń po przywróceniu ustawień fabrycznych oraz zebranie metadanych o aplikacjach zainstalowanych domyślnie, ze szczególnym uwzględnieniem możliwości kontroli wykorzystania danych sieciowych w tle.

Sprzęt: dwa telefony Samsung Galaxy S7 (po resecie do ustawień fabrycznych).

Po oczyszczeniu zrzutów z urządzeń uzyskano spójny zbiór **53** aplikacji domyślnych na każdym z telefonów.

Wyniki zbiorcze:

TABELA 5.1: Podsumowanie ustawień korzystania z danych sieciowych w tle (po resecie)

Kategoria	Liczba	Udział
Łączna liczba aplikacji	53	100%
Można zablokować dane w tle	44	83,0%
Nie można zablokować	9	17,0%
Domyślnie aktywny transfer	53	100%
Domyślnie zablokowany	0	0%

Legenda: „Można zablokować dane w tle” – aplikacje, dla których system Android udostępnia opcję ograniczenia transmisji danych w tle; „Nie można zablokować” – aplikacje i komponenty systemowe zachowujące dostęp do sieci w tle niezależnie od użytkownika; „Domyślnie aktywny transfer/zablokowany” – stan tuż po resecie fabrycznym; „Udział” – procent względem wszystkich zidentyfikowanych aplikacji ($N = 53$).

Aplikacje z nieograniczonym dostępem do danych w tle:

TABELA 5.2: Lista aplikacji, dla których nie można zablokować danych sieciowych w tle (po resecie)

Nazwa aplikacji
Konserwacja urządzenia
Samsung Cloud
Samsung Members
Sim AT
SmartThings
Udostępnianie łącza
Ustawienia
Workspace

Wniosek: stan „po resece” obejmuje wyłącznie aplikacje *domyślnie aktywne*, z których zdecydowaną większość (83%) można ręcznie ograniczyć pod kątem wykorzystania danych w tle. Około 17% stanowią jednak komponenty, które zachowują pełny dostęp do sieci w tle i nie podlegają tej kontroli.

Uwaga replikacyjna: powyższe wyniki pochodzą z pojedynczego zrzutu po resece na tym samym systemie operacyjnym w obu urządzeniach.

5.2 Przechwytywanie radiowe Wiresharkiem w trybie monitorowania (niepowodzenie)

Cel: pasywna obserwacja ruchu bez konfigurowania proxy na urządzeniu.

Sprzęt: laptop z Linuksem; karta Wi-Fi w trybie monitorowania; dwa Galaxy S7.

Metoda: nasłuch (monitor mode) podczas korzystania z Wi-Fi zabezpieczonego (WPA2), próba późniejszej dekrypcji ramek i analizy ruchu aplikacji.

Wynik: *niepowodzenie metody* w kontekście pełnej analizy treści: kluczowy ruch pozostaje szyfrowany (WPA2), brak materiału do bezstratnej dekrypcji (m.in. konieczność złapania handshake i posiadania parametrów sieci). Metoda okazała się niewspółmiernie pracochłonna i ograniczona względem celów projektu; zrezygnowano na rzecz podejścia proxy/pcap.

Wnioski: przy analizie aplikacji mobilnych efektywniejsze jest środowisko kontrolowane (AP + proxy/pcap) niż sniffing radiowy na produkcyjnej sieci.

5.3 MITM przez proxy

Cel: przechwytywanie i inspekcja ruchu HTTP(S) generowanego przez badane aplikacje.

Narzędzia: *mitmproxy*, *tcpdump*.

Sprzęt i sieć: laptop z systemem Debian skonfigurowany jako punkt dostępowy (hostapd + dnsmasq) oraz dwa telefony Samsung Galaxy S7 po resece do ustawień fabrycznych.

Metoda:

1. Konfiguracja AP i routingu; uruchomienie *mitmproxy* w trybie przechwytyującym oraz równoległe rejestrowanie pakietów za pomocą *tcpdump*.
2. Instalacja certyfikatu użytkownika *mitmproxy* na urządzeniach oraz próby uruchomienia i obsługi wybranych aplikacji.

Wynik: Przy podejściu pierwszym aplikacje odmawiały działania, wykrywając obecność proxy i blokując komunikację. Próba drugiego wariantu (instalacja certyfikatu i obejście pinningu) również zakończyła się niepowodzeniem: telefony odrzucały certyfikat podczas pracy, a dodatkowo podczas próby uzyskania uprawnień *root* w celu głębszej ingerencji doszło do uszkodzenia jednego z urządzeń. Sytuacja ta wymusiła zmianę metodologii i poszukiwanie alternatywnych rozwiązań.

Wnioski: samo zastosowanie proxy MITM daje bardzo ograniczoną obserwowalność wobec aplikacji stosujących pinning i/lub protokół QUIC. Niezbędne są inne podejścia, np. instrumentacja, przechwytywanie ruchu na poziomie systemowym (pcap) lub konwersja ruchu HTTP/3 na HTTP/2.

5.4 MITM z instrumentacją (Frida) wobec pinningu — próba nieudana

Cel: obejście mechanizmu *certificate pinning* i przywrócenie możliwości inspekcji treści żądań.

Narzędzia: *Frida* (*frida-server* oraz gotowe skrypty hookujące interfejsy API związane z walidacją certyfikatów).

Metoda: uruchomiono *frida-server* na urządzeniu i wstrzyknięto skrypty mające na celu przechwycenie wywołań metod weryfikujących certyfikaty oraz ich dynamiczne obejście.

Wynik: próba zakończyła się *niepowodzeniem*. Wstrzyknięte skrypty nie zadziałały w przypadku analizowanej aplikacji. Prawdopodobnymi przyczynami były: brak uprawnień *root*, aktywne mechanizmy SELinux oraz natywne implementacje weryfikacji certyfikatów (np. niestandardowe *TrustManager*), których nie obejmowały standardowe hooki.

Wnioski: w wielu nowoczesnych aplikacjach obejście pinningu bez dostępu do uprawnień *root* jest nierealne lub bardzo trudne. Alternatywami pozostają uruchamianie aplikacji w środowisku z pełnym dostępem administracyjnym (emulator, urządzenie z *root*) lub analiza ograniczona do metadanych transportowych (hosty, porty, czasy, wolumen danych), bez wglądu w same treści komunikatów.

5.5 Rejestracja ruchu po stronie urządzenia (PCAP/PCAPNG) — przykład Instagram (sukces)

Cel: charakterystyka ruchu bez naruszania zabezpieczeń aplikacji (rejestracja na urządzeniu).

Metoda: rejestracja ruchu i analiza strumieni; w tej próbie dominował HTTP/3/QUIC (UDP/443) z kanałem utrzymania sesji po TCP/5222, a ruch HTTP/1.1/HTTPS był śladowy.

Wynik: metoda skuteczna do obserwacji protokołów, kierunków i aktywności w tle; semantyka HTTP ograniczona przez QUIC/pinning. **Artefakty:** patrz rozdział z eksperymentem głównym (plik *.pcapng* i agregaty).

Rozdział 6

Eksperyment: analiza ruchu aplikacji Instagram z użyciem PCAPdroid

6.1 Cel i założenia

Celem eksperymentu jest empiryczna charakterystyka komunikacji sieciowej popularnej aplikacji mobilnej (Instagram) pod kątem: (i) wykorzystywanych protokołów i transportu (HTTP/1.1, HTTP/2, HTTP/3/QUIC, TLS) [10], [22], [28], (ii) wzorców zachowania (bursty podczas interakcji vs. aktywność w tle), (iii) rozproszenia docelowych usług (domen/IP/portów) oraz (iv) ograniczeń obserwowalności wynikających z QUIC/HTTP/3 i ewentualnego *certificate pinning*. Rejestracji dokonano narzędziem PCAPdroid (no-root, tryb PCAPNG z kluczami) i poddano obróbce wg metodyki z Rozdz. ??; do inspekcji pcapng wykorzystywano również Wireshark [31].

6.2 Środowisko i narzędzia

- Urządzenie z Androidem, konto testowe; sieć laboratoryjna (AP na Linuxie).
- **PCAPdroid**: lokalny VPN (bez roota), *TLS Decryption* aktywne, zapis do .pcapng (z kluczami sesji).
- Analiza pcapng: Wireshark/tshark (ALPN, SNI, QUIC/UDP 443, identyfikacja HTTP/1.1/2/3) [10], [22], [31].

6.3 Procedura (pojedyncza iteracja)

Zgodnie z metodyką, jedna iteracja (30–34 min) składa się z:

1. **Baseline OS (2 min)** – app nieuruchomiona; punkt odniesienia dla tła.
2. **Cold start (1 min)** – uruchomienie i ustabilizowanie ekranu głównego.
3. **Logowanie (3 min, jeśli dotyczy)** – ścieżki auth/token.
4. **Nawigacja (5 min)** – główne ekrany (feed, profil, wyszukiwarka).
5. **Media/operacja (3 min)** – krótkie odtwarzanie/przewijanie, wymuszenie większych transferów.
6. **Offline/online (4 min)** – 2 min offline + 2 min po reconnect; obserwacja retry/backoff.

7. **Zamknięcie (1 min)** – wylogowanie/zamknięcie.

8. **Tło (15 min)** – 5 min tuż po zejściu do tła + 10 min dalszej obserwacji.

Motywacja czasów i uzasadnienia znajdują się w Rozdz. ???. QUIC/HTTP/3 (UDP) z natury ogranicza klasyczny MITM; analiza treści HTTP możliwa jest przede wszystkim w wariancie porównawczym z wymuszeniem *fallbacku* do HTTP/2/TCP (blokada UDP/443) [10].

6.4 Zebrane dane (ta sesja)

Eksperyment wykonano w dniu 2025-08-18 w oknie **17:56:33–18:10:14** (13.7 min). Zarejestrowano **302** przepływy (*flows*) o łącznym wolumenie **28.0 MiB**; **RX 27.2 MiB**, **TX 0.85 MiB**.

Udział protokołów (liczba flow): QUIC **61.9%**, DNS 16.2%, TLS 11.9%, TCP 4.0%, HTTP 2.6%, HTTPS 1.7%, UDP 1.7%. Dominującym portem docelowym był **443** (HTTPS/QUIC; 233 flow), dalej **53** (DNS; 49 flow), **5222** (TCP; 12 flow) i **80** (HTTP; 8 flow).

6.5 Wyniki ilościowe

6.5.1 Wolumen i liczność wg protokołu

TABELA 6.1: Podsumowanie wg protokołu (Instagram, ta sesja)

Protokół	Flows	MiB (total)	B wysłane	B odebrane
QUIC	187	27.99	855,220	28,500,436
HTTPS	5	0.02	12,180	9,063
TCP	12	0.02	15,384	3,699
DNS	49	0.00	1,475	2,441
TLS	36	0.00	2,160	1,440
UDP	5	0.00	837	346
HTTP	8	0.00	480	320
Razem	302	28.04	888,736	28,517,745

6.5.2 Największe cele (IP/port) wg wolumenu

TABELA 6.2: Największe kierunki ruchu (wolumen; Instagram, ta sesja)

IP docelowe	Port	Proto	Flows	MiB (total)
185.60.217.63	443	QUIC	137	27.52
157.240.251.63	443	QUIC	1	0.33
185.60.217.3	443	QUIC	35	0.08
185.60.217.20	443	QUIC	2	0.02
185.60.217.175	5222	TCP	11	0.02
157.240.251.11	443	QUIC	1	0.02
185.60.217.28	443	QUIC	1	0.01
157.240.0.63	443	QUIC	1	0.01
185.60.217.13	443	HTTPS	2	0.01
157.240.253.17	443	HTTPS	1	0.01

6.6 Obserwacje jakościowe

- **Dominacja HTTP/3/QUIC.** Zdecydowana większość ruchu (liczebnie i wolumenowo) to QUIC na 443/UDP, co jest zgodne z trendem migracji warstwy mediów do HTTP/3 [10], [22].
- **Kanał utrzymania sesji.** Występuje równoległy kanał na **TCP/5222** (11–12 flows, mały wolumen), prawdopodobnie do komunikacji w czasie rzeczywistym/utrzymania połączenia.
- **Burst podczas interakcji, potem „oddech”.** Zliczenia per minuta (niezamieszczone tu dla zwięzłości) pokazują silny pik około 17:59 (setki nowych/kończących się flow), po którym następują krótkie sekwencje aktywności w tle (18:05–18:06).
- **Niewielki ślad HTTP/1.1/HTTPS.** Jedynie kilka żądań; główna komunikacja i zasoby idą ścieżką QUIC.

6.7 Dyskusja i ograniczenia

Obserwowalność treści. Ze względu na transport QUIC/UDP, klasyczny MITM dla TLS (po TCP) nie ma zastosowania; nawet z certyfikatem użytkownika wgląd w treści HTTP jest ograniczony [10]. Analiza opiera się na metadanych (ALPN, SNI, strumienie, wolumeny, czasy) i korelacji zachowania.

Pinning. Nawet przy braku aktywnie zaimplementowanego pinningu, wybór QUIC ogranicza możliwość przechwycenia semantyki HTTP. W aplikacjach z rygorystycznym *certificate pinning* obserwowalność jest dodatkowo zawężona do metadanych (zob. [12]).

Replikowalność i zmienność. Wyniki dotyczą konkretnych wersji aplikacji i backendu, daty 2025-08-18, okna 13.7 min i scenariusza przewijania feedu.

6.8 Wariant porównawczy (HTTP/3 → HTTP/2)

Aby uzyskać wgląd w semantykę HTTP, zaplanowano wariant kontrolny z wymuszeniem *fallbacku* z QUIC na HTTP/2/TCP (blokada UDP/443 w sieci labowej) [10]. Porównywane będą: liczba żądań, rozkład kodów i metod HTTP (gdy odszyfrowalne), TTFB/czasy transferu, liczba i rola domen docelowych.

6.9 Artefakty i replikowalność

Zachowano .pcapng (plik zawiera zarówno pakiety, jak i klucze sesji), dziennik przebiegu (czas kroków), listę wersji aplikacji/OS oraz wyciągi agregatów (tabele 6.1 i 6.2). Schemat nazewnictwa plików: YYYYMMDD_HHMM_Instagram_runN_net={default|udp443_blocked}_vApp_vAndroid.pcapng.

6.10 Podsumowanie

Eksperyment pokazuje typowy dla nowoczesnych aplikacji wzorzec: dominację HTTP/3/QUIC dla treści (media), lekki kanał sterujący po TCP/5222 oraz okresowe aktywności w tle. W kontekście prywatności i bezpieczeństwa oznacza to, że analiza semantyczna wymaga wariantów kontrolnych (H3→H2) lub metod instrumentacyjnych; jednocześnie metadane ruchu pozwalają na wiarygodną charakterystykę wzorców komunikacyjnych i rozproszenia dostawców.

Rozdział 7

Aspekty bezpieczeństwa i prywatności

7.1 Szyfrowanie i wersje TLS

Wykryte wersje, renegotjacje (jeśli obserwowane).

7.2 Certificate pinning

Przypadki utraty widoczności ruchu (fail-open / fail-closed), wpływ na wyniki.

7.3 Potencjalne wycieki metadanych

Parametry w URL, identyfikatory urządzeń, tokeny.

7.4 Domeny trackingowe

Porównanie intensywności ruchu do domen analitycznych.

7.5 Implikacje

Ryzyka dla prywatności użytkownika i rekomendacje (np. ograniczanie uprawnień sieciowych, firewall lokalny).

Rozdział 8

Dyskusja

8.1 Interpretacja wyników

W jakim stopniu dane potwierdzają/obalają hipotezy.

8.2 Porównanie z literaturą

Zestawienie różnic (np. wzrost QUIC vs wcześniejsze badania).

8.3 Zagrożenia dla ważności

Internal: artefakty środowiska testowego; external: różnice regionów geograficznych; konstrukcja: wybór kategorii.

8.4 Perspektywa praktyczna

Wartość dla użytkowników, deweloperów, regulatorów.

Rozdział 9

Wnioski i dalsze prace

9.1 Podsumowanie

Najważniejsze osiągnięcia i wkład.

9.2 Odpowiedzi na pytania badawcze

Zwięzłe odpowiedzi w punktach.

9.3 Możliwe rozszerzenia

Automatyczna detekcja anomalii, uczenie maszynowe do klasyfikacji domen, analiza korelacji z zużyciem energii.

Bibliografia

- [1] Android Open Source Project. „Application Sandbox.” (2025), adr.: <https://source.android.com/docs/security/app-sandbox> (term. wiz. 17.08.2025).
- [2] A. Razaghpanah, R. Nithyanand, N. Vallina-Rodriguez i in., „Apps, Trackers, Privacy, and Regulators: A Global Study of the Mobile Tracking Ecosystem,” w *Network and Distributed System Security Symposium (NDSS)*, 2018. adr.: https://www.ndss-symposium.org/wp-content/uploads/2018/02/ndss2018_05B-3_Razaghpanah_paper.pdf.
- [3] Android Open Source Project. „Architecture overview.” (2025), adr.: <https://source.android.com/docs/core/architecture> (term. wiz. 17.08.2025).
- [4] „Cisco Annual Internet Report (2018–2023) White Paper,” Cisco, 2020, Prognozy do 2023 r.; m.in. penetracja subskrypcji mobilnych (71% global population with mobile subscription by 2023). adr.: <https://www.cisco.com/c/en/us/solutions/collateral/executive-perspectives/annual-internet-report/white-paper-c11-741490.html> (term. wiz. 17.08.2025).
- [5] *Exodus Privacy: Analysis of trackers and permissions in Android apps*, <https://exodus-privacy.eu.org/>, Accessed 2025-08-17, 2025.
- [6] *Frida: A world-class dynamic instrumentation toolkit*, <https://frida.re/docs/home/>, Accessed 2025-08-17, 2025.
- [7] StatCounter, *Global Platform Market Share: Desktop vs Mobile vs Tablet*, Udział globalnych page views według platformy; potwierdza >50% dla urządzeń mobilnych (sprawdź dokładny zakres dat), 2025. adr.: <https://gs.statcounter.com/platform-market-share/desktop-mobile-tablet> (term. wiz. 17.08.2025).
- [8] Google. „How We Kept the Google Play and Android App Ecosystem Safe in 2024.” Android app ecosystem security overview. (sty. 2025), adr.: <https://security.googleblog.com/2025/01/how-we-kept-google-play-android-app-ecosystem-safe-2024.html> (term. wiz. 17.08.2025).
- [9] *HTTP/2*, RFC 9113, 2022. adr.: <https://www.rfc-editor.org/rfc/rfc9113>.
- [10] M. Bishop, *HTTP/3*, RFC 9114, 2022. adr.: <https://www.rfc-editor.org/rfc/rfc9114>.
- [11] OWASP Mobile Security Testing Guide. „MASTG-TECH-0120: Intercepting HTTP Traffic Using an Interception Proxy.” (2025), adr.: <https://mas.owasp.org/MASTG/techniques/generic/MASTG-TECH-0120/> (term. wiz. 17.08.2025).
- [12] OWASP Mobile Security Testing Guide. „MASTG-TEST-0244: Missing Certificate Pinning in Network APIs.” (2025), adr.: <https://mas.owasp.org/MASTG/tests/android/MASVS-NETWORK/MASTG-TEST-0244/> (term. wiz. 17.08.2025).
- [13] *mitmproxy: Interactive TLS-capable intercepting HTTP proxy*, <https://mitmproxy.org/>, Accessed 2025-08-17, 2025.

- [14] mitmproxy project. „mitmproxy — About Certificates.” (2025), adr.: <https://docs.mitmproxy.org/stable/concepts/certificates/> (term. wiz. 17.08.2025).
- [15] *Mobile Security Framework (MobSF) Documentation*, <https://github.com/MobSF/docs>, Accessed 2025-08-17, 2025.
- [16] Android Developers. „Permissions on Android — Overview.” (2025), adr.: <https://developer.android.com/guide/topics/permissions/overview> (term. wiz. 17.08.2025).
- [17] Android Developers. „Platform architecture.” (2024), adr.: <https://developer.android.com/guide/platform> (term. wiz. 17.08.2025).
- [18] J. M. del Álamo, J. López-Fernández i in., „Privacy Assessment in Android Apps: A Systematic Mapping Study,” *Electronics*, t. 10, nr. 16, s. 1999, 2021. adr.: <https://www.mdpi.com/2079-9292/10/16/1999>.
- [19] Google. „Privacy Sandbox on Android — Overview.” (2025), adr.: <https://privacysandbox.google.com/overview/android> (term. wiz. 17.08.2025).
- [20] *Profile battery usage with Batterystats and Battery Historian*, <https://developer.android.com/topic/performance/power/setup-battery-historian>, Accessed 2025-08-17, 2024.
- [21] *Profile your app performance (Android Studio Profiler)*, <https://developer.android.com/studio/profile>, Accessed 2025-08-17, 2024.
- [22] *QUIC: A UDP-Based Multiplexed and Secure Transport*, RFC 9000, 2021. adr.: <https://www.rfc-editor.org/rfc/rfc9000>.
- [23] Android Developers. „Request runtime permissions.” (2025), adr.: <https://developer.android.com/training/permissions/requesting> (term. wiz. 17.08.2025).
- [24] V. F. Taylor, R. Spolaor, M. Conti i I. Martinovic, „Robust Smartphone App Identification via Encrypted Network Traffic Analysis,” *arXiv preprint arXiv:1704.06099*, 2017. adr.: <https://arxiv.org/pdf/1704.06099>.
- [25] Android Developers. „SdkSandboxManager API reference.” (2025), adr.: <https://developer.android.com/design-for-safety/privacy-sandbox/reference/sdksandbox/SdkSandboxManager> (term. wiz. 17.08.2025).
- [26] *tcpdump/libpcap*, <https://www.tcpdump.org/>, Accessed 2025-08-17, 2025.
- [27] M. Conti, Q. Li, A. Maragno i R. Spolaor, „The Dark Side(-Channel) of Mobile Devices: A Survey on Network Traffic Analysis,” *arXiv preprint arXiv:1708.03766*, 2017. adr.: <https://arxiv.org/abs/1708.03766>.
- [28] E. Rescorla, *The Transport Layer Security (TLS) Protocol Version 1.3*, RFC 8446, 2018. adr.: <https://www.rfc-editor.org/rfc/rfc8446>.
- [29] A. Pathak, Y. C. Hu i M. Zhang, „Where is the Energy Spent in Android Apps? Fine-Grained Energy Accounting on Smartphones with EProf,” w *Proceedings of the 7th ACM European Conference on Computer Systems (EuroSys '12)*, ACM, 2012.
- [30] S. Fahl, M. Harbach, T. Muders, L. Baumgärtner, B. Freisleben i M. Smith, „Why Eve and Mallory Love Android: An Analysis of Android SSL (In)Security,” w *Proceedings of the 2012 ACM Conference on Computer and Communications Security (CCS '12)*, ACM, 2012. adr.: <https://saschafahl.de/static/paper/androidssl2012.pdf>.
- [31] Wireshark Foundation. „Wireshark User’s Guide.” (2025), adr.: <https://www.wireshark.org/docs/> (term. wiz. 17.08.2025).

(2025)

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu L^AT_EX (Overleaf).